

Engineering Design Document: Project Aegis

A Distributed, Multi-Tenant, Content-Addressable Storage and Synchronization Engine

Document Track: Core Systems Architecture | Status: Approved for Implementation | Classification: Technical Blueprint

1. Executive Summary & Design Invariants

Project Aegis is an exabyte-capable, cloud-agnostic file storage and synchronization platform modeled on the operational mechanics of distributed file platforms (e.g., Google Drive, Dropbox Magic Pocket).

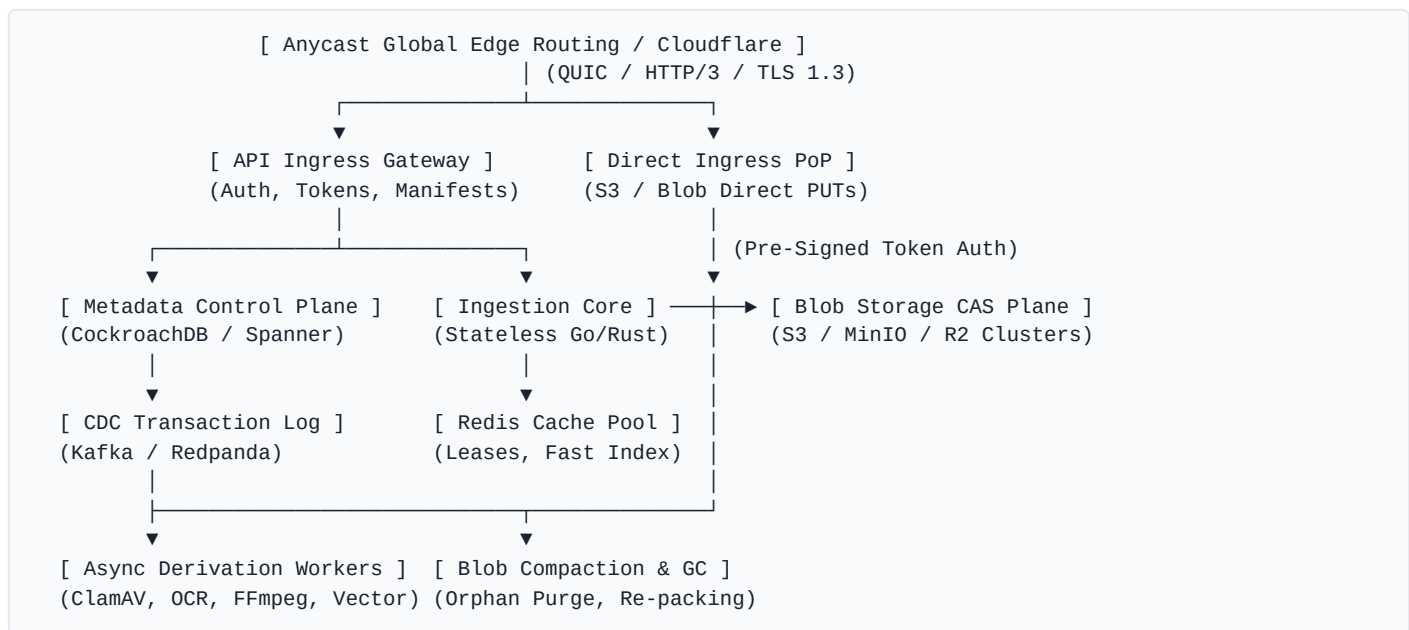
The platform separates the **unstructured binary storage plane (Data Plane)** from the **hierarchical relational state and access control engine (Control/Metadata Plane)**. This physical and logical decoupling ensures that high-throughput binary I/O operations cannot exhaust database connection pools or starve metadata execution threads.

SYSTEM INVARIANTS		
Bit-Perfect CAS Data payloads are addressed exclusively by cryptographic hash; mutations are strictly append-only.	Acyclic Namespace The filesystem graph is an immutable DAG; directory cycles and broken lineage paths are rejected at schema.	Cryptographic Ingress Direct-to-blob access requires short-lived, tenant-scoped HMAC tokens evaluated at edge storage endpoints.

Core System Guarantees

- Zero-Byte Ingress Corruption:** Chunks are hashed on client endpoints before transit using Content-Defined Chunking (FastCDC) and re-verified on ingestion. An upload is only acknowledged (201 Created) when block checksums match the storage manifest.
- Deterministic Metadata Linearizability:** Metadata updates execute inside ACID transaction boundaries. Ancestor permissions and directory hierarchies use materialized path models to eliminate recursive graph traversals.
- Decoupled Derivation Boundaries:** Auxiliary derivation workloads (e.g., ClamAV scanning, optical character recognition, thumbnail rendering, metadata extraction) run out-of-band via distributed log streams, preventing pipeline delays from stalling the core upload path.

2. End-to-End System Topology



Data Plane Separation

- **API Control Plane:** Terminates client session handshakes, reconciles block presence in the CAS registry, provisions ephemeral signed upload channels, and commits completed version manifests.
- **Ingress Data Plane:** High-throughput block ingress bypassing web application runtimes. Payload bytes stream directly to durable, content-addressable object storage nodes via edge acceleration.
- **Asynchronous Message Bus:** Redpanda/Kafka message brokers capture metadata log changes (Change Data Capture) and distribute compute jobs across stateless worker fleets.

3. Relational Control Plane & Schema Architecture

The relational schema uses PostgreSQL/CockroachDB with the `ltree` extension to model directory hierarchies, tracking hierarchical paths while maintaining $O(1)$ cycle detection and localized subtree updates.

```

-- Workspace & Tenant Scoping
CREATE TABLE tenants (
    tenant_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    kms_key_arn VARCHAR(255) NOT NULL,
    storage_quota_bytes BIGINT NOT NULL,
    used_bytes BIGINT DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Hierarchical Directory Graph
CREATE TYPE node_type AS ENUM ('FILE', 'DIRECTORY');

CREATE TABLE namespace_nodes (
    node_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(tenant_id) ON DELETE CASCADE,
    parent_id UUID REFERENCES namespace_nodes(node_id),
    name VARCHAR(255) NOT NULL,
    type node_type NOT NULL,

    -- Materialized ancestor path representation: "root_uuid.parent_uuid.node_uuid"
    lineage_path LTREE NOT NULL,
    acl_epoch BIGINT NOT NULL DEFAULT 1,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT uq_parent_name UNIQUE (tenant_id, parent_id, name, is_deleted)
);

CREATE INDEX idx_namespace_lineage ON namespace_nodes USING GIST (lineage_path);
CREATE INDEX idx_namespace_parent ON namespace_nodes (tenant_id, parent_id);

-- Immutable File Revisions
CREATE TABLE file_versions (
    version_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    node_id UUID NOT NULL REFERENCES namespace_nodes(node_id) ON DELETE CASCADE,
    version_number INT NOT NULL,
    total_size_bytes BIGINT NOT NULL,
    mime_type VARCHAR(128) NOT NULL,
    content_sha256 BYTEA NOT NULL,
    created_by UUID NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT uq_node_version UNIQUE (node_id, version_number)
);

-- Content-Addressable Storage (CAS) Block Registry
CREATE TABLE cas_blocks (
    block_hash BYTEA PRIMARY KEY, -- 32-byte binary SHA-256
    tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
    size_bytes INT NOT NULL,
    storage_tier VARCHAR(32) NOT NULL DEFAULT 'HOT',
    ref_count BIGINT NOT NULL DEFAULT 1,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Ordered Merkle Manifest
CREATE TABLE file_manifest_blocks (
    version_id UUID NOT NULL REFERENCES file_versions(version_id) ON DELETE CASCADE,
    chunk_index INT NOT NULL,
    block_hash BYTEA NOT NULL REFERENCES cas_blocks(block_hash),
    offset_bytes BIGINT NOT NULL,
    size_bytes INT NOT NULL,

    PRIMARY KEY (version_id, chunk_index)
);

-- Ephemeral Ingress Sessions

```

```
CREATE TABLE upload_sessions (
    session_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(tenant_id),
    node_id UUID REFERENCES namespace_nodes(node_id),
    total_size BIGINT NOT NULL,
    expected_chunks INT NOT NULL,
    chunks_received INT NOT NULL DEFAULT 0,
    expires_at TIMESTAMPTZ NOT NULL,
    is_completed BOOLEAN NOT NULL DEFAULT FALSE
);
```

Stored Procedure: Transactional, Cycle-Free Directory Moves

```
CREATE OR REPLACE FUNCTION move_directory(
    p_tenant_id UUID,
    p_node_id UUID,
    p_new_parent_id UUID
) RETURNS VOID AS $$
DECLARE
    v_src_path LTREE;
    v_dst_path LTREE;
    v_node_name VARCHAR;
BEGIN
    -- 1. Acquire pessimistic row locks to prevent concurrent tree modifications
    SELECT lineage_path, name INTO v_src_path, v_node_name
    FROM namespace_nodes
    WHERE node_id = p_node_id AND tenant_id = p_tenant_id
    FOR UPDATE;

    SELECT lineage_path INTO v_dst_path
    FROM namespace_nodes
    WHERE node_id = p_new_parent_id AND tenant_id = p_tenant_id
    FOR SHARE;

    -- 2. Reject cyclic assignments: Target path cannot be a descendant of Source path
    IF v_dst_path <@ v_src_path THEN
        RAISE EXCEPTION 'Cyclic hierarchy violation: Target is a descendant of Source.';
    END IF;

    -- 3. Atomic Path Remapping across all subtree descendants
    UPDATE namespace_nodes
    SET
        parent_id = CASE WHEN node_id = p_node_id THEN p_new_parent_id ELSE parent_id END,
        lineage_path = v_dst_path || text2ltree(replace(p_node_id::text, '-', '_')) || subpath(lineage_path, n
        acl_epoch = acl_epoch + 1,
        updated_at = NOW()
    WHERE lineage_path <@ v_src_path AND tenant_id = p_tenant_id;
END;
$$ LANGUAGE plpgsql;
```

4. Content-Defined Chunking Engine (Rust FastCDC)

The client and ingress workers use a Content-Defined Chunking engine (`FastCDC`) based on Gear-hashing and two-phase mask normalization. This shifts chunk boundaries dynamically around modifications, preventing byte-shift boundary corruption in large binary streams.

```

use sha2::{Digest, Sha256};
use std::io::{self, Read};

const MIN_CHUNK: usize = 64 * 1024;          // 64 KB Lower Boundary
const AVG_CHUNK: usize = 1024 * 1024;        // 1 MB Normalized Target
const MAX_CHUNK: usize = 4 * 1024 * 1024;    // 4 MB Upper Boundary
const MASK_S: u32 = 0x0003_FFFF;             // Sub-average threshold mask
const MASK_L: u32 = 0x0007_FFFF;             // Post-average threshold mask

static GEAR_MATRIX: [u32; 256] = [
    0x8a88094a, 0x0036734c, 0xb686c450, 0xa887f46d, 0xb036136e, 0xb0e9acdd, 0x1f92e6ba, 0x909f24fe,
    0x7f4c641d, 0x46492866, 0xae07d77b, 0x4784d83b, 0x54098690, 0x6241a78e, 0xd887929e, 0x5113e9e2,
    0x540e3079, 0x2775883f, 0x851499b9, 0x12a9ce0b, 0x5594f8b6, 0x4622b39f, 0x924d5254, 0x7b266f96,
    0x6f736182, 0xd5992920, 0x28417f46, 0xb37115a2, 0x6a24d00f, 0x2e737b0e, 0xd0be4b15, 0x22047a24,
    0x9026742c, 0x42be677b, 0x2c45665f, 0x809fb52b, 0x767d28d8, 0x59be3a6c, 0x60012224, 0xa45e0241,
    0xd0445513, 0xf8421033, 0xe10e6a1f, 0x6380ff96, 0x73a0dd89, 0x76233014, 0x6984e528, 0xd6134137,
    0x08796842, 0x78696955, 0x64284273, 0xb00e2b9c, 0x8cbb65b7, 0xbe0e7d52, 0x46e472d7, 0xa66b264f,
    0x306a3974, 0xb79c3028, 0xaa2c6677, 0xaf8f7ec5, 0xd838ca69, 0x43f0084f, 0x235e98be, 0xb8be115c,
    0x3f5625d9, 0xe00d95da, 0xb0cf29e3, 0x3be456ad, 0x2ab97f47, 0x3cc09c04, 0x264b7d8c, 0x7ae85865,
    0x67c342d2, 0x469241fe, 0x4b192304, 0xf159751e, 0x56bedea0, 0x30935ab9, 0x8c722930, 0x0093319e,
    0x87f753ee, 0x7d02dd5d, 0x8a51663a, 0x12344040, 0x99214b15, 0x43bda7da, 0x4189e783, 0x2f6be255,
    0x46487040, 0xe41310b1, 0x6de2515b, 0x8f619944, 0x0a4dafd7, 0xd77076a3, 0x491f53c4, 0x7591b169,
    0x48cad47f, 0x2614b18b, 0xf64f4342, 0xaeb16770, 0x00402d6a, 0xebf263b4, 0x40d1217e, 0x7a237cb3,
    0x5d2417b3, 0x93c9a0cf, 0x2cc89021, 0x3f7a4bab, 0xe840b972, 0x162a69fd, 0xb98a3794, 0x8349f829,
    0xf2a34b29, 0x8e622228, 0xb8420014, 0x208479d3, 0x15f2d01e, 0x600237a9, 0xd40279be, 0x001a4060,
    0x290005a0, 0x96a2104d, 0xac61a7be, 0x99483fd6, 0x22605439, 0xbe037d0e, 0x3564aa61, 0x4c2696ee,
    0xb3401321, 0xaecb527d, 0x00b07a23, 0xa5bc5312, 0x3209887b, 0xb540029f, 0x70031105, 0xa0089530,
    0x3613076e, 0x4a0b22d1, 0x22a4ff82, 0x40020853, 0xb80091dc, 0xbefd9376, 0x07f77f2c, 0x42c0bdcc,
    0x53089408, 0xeb35497b, 0x5cca466d, 0xd20a402d, 0x422a8724, 0xb1603598, 0x11021fae, 0x1e40605a,
    0xd0512702, 0x4ca124e8, 0x35da4750, 0x2b40ff30, 0x52138e83, 0x91007351, 0xeeab4f62, 0x3c1b291e,
    0x91494390, 0x574fc762, 0x42b6a41f, 0x200e4fe0, 0x6bfe5280, 0x807a750d, 0x1500003b, 0x4e6d8020,
    0x728349f8, 0x20402405, 0x0640a896, 0xa4315277, 0xa2a4f1e8, 0xf0222205, 0x8200a609, 0x40081255,
    0xfbe8235b, 0x26ae2842, 0x338241a7, 0x7316b0b1, 0x726402f4, 0x002b0e00, 0x4300bb7b, 0x40a649f0,
    0x8a8a2062, 0x4b001605, 0x620099ee, 0x84000611, 0x10222005, 0x47620021, 0x70520017, 0x3f5b0011,
    0x71000010, 0x00200000, 0x00000000, 0x00000020, 0x00000200, 0x00002000, 0x00020000, 0x00200000,
    0x02000000, 0x20000000, 0x00000001, 0x00000010, 0x00000100, 0x00001000, 0x00010000, 0x00100000,
    0x01000000, 0x10000000, 0x00000002, 0x00000004, 0x00000008, 0x00000040, 0x00000080, 0x00000400,
    0x00000800, 0x00004000, 0x00008000, 0x00040000, 0x00080000, 0x00400000, 0x00800000, 0x04000000,
    0x08000000, 0x40000000, 0x80000000, 0x7fffffff, 0xbfffffff, 0xdfffffff, 0xffffffff, 0xf7ffffff,
    0xfbffffff, 0xfdfbffff, 0xfeffff, 0xfffbffff, 0xffdfffff, 0xffefffff, 0xffff7fff,
    0xffffbfff, 0xffffdfff, 0xffffefff, 0xfffff7ff, 0xfffffbff, 0xfffffdff, 0xffffffef, 0xfffffff7,
];

pub struct ChunkDescriptor {
    pub hash: [u8; 32],
    pub offset: u64,
    pub length: usize,
}

pub struct FastCDCStream<R: Read> {
    reader: R,
    buffer: Vec<u8>,
    global_offset: u64,
}

impl<R: Read> FastCDCStream<R> {
    pub fn new(reader: R) -> Self {
        Self {
            reader,
            buffer: Vec::with_capacity(MAX_CHUNK * 2),
            global_offset: 0,
        }
    }

    pub fn next_chunk(&mut self) -> io::Result<Option<ChunkDescriptor>> {
        while self.buffer.len() < MAX_CHUNK {
            let mut temp = [0u8; 64 * 1024];

```

```

        let bytes_read = self.reader.read(&mut temp)?;
        if bytes_read == 0 { break; }
        self.buffer.extend_from_slice(&temp[..bytes_read]);
    }

    if self.buffer.is_empty() {
        return Ok(None);
    }

    let chunk_len = self.compute_cut_point(self.buffer.len().min(MAX_CHUNK));
    let chunk_slice = &self.buffer[..chunk_len];

    let mut hasher = Sha256::new();
    hasher.update(chunk_slice);
    let hash: [u8; 32] = hasher.finalize().into();

    let descriptor = ChunkDescriptor {
        hash,
        offset: self.global_offset,
        length: chunk_len,
    };

    self.global_offset += chunk_len as u64;
    self.buffer.drain(..chunk_len);

    Ok(Some(descriptor))
}

#[inline(always)]
fn compute_cut_point(&self, max_index: usize) -> usize {
    if max_index <= MIN_CHUNK {
        return max_index;
    }

    let mut fingerprint: u32 = 0;
    let mid_point = AVG_CHUNK.min(max_index);

    // Sub-average threshold matching
    for i in MIN_CHUNK..mid_point {
        let byte = self.buffer[i];
        fingerprint = (fingerprint << 1).wrapping_add(GEAR_MATRIX[byte as usize]);
        if (fingerprint & MASK_S) == 0 {
            return i + 1;
        }
    }

    // Normalized tail matching
    for i in mid_point..max_index {
        let byte = self.buffer[i];
        fingerprint = (fingerprint << 1).wrapping_add(GEAR_MATRIX[byte as usize]);
        if (fingerprint & MASK_L) == 0 {
            return i + 1;
        }
    }

    max_index
}
}

```

5. Ingress Orchestration & Manifest Processing (Go Core)

The stateless Go ingress coordinator processes incoming upload manifests, filters existing content hashes to reduce redundant transfers, issues signed upload URLs, and commits versions to PostgreSQL.

```

package main

import (
    "context"
    "crypto/hmac"
    "crypto/sha256"
    "database/sql"
    "encoding/hex"
    "encoding/json"
    "fmt"
    "net/http"
    "time"

    "github.com/google/uuid"
    _ "github.com/jackc/pgx/v5/stdlib"
)

type ChunkManifest struct {
    Index      int    `json:"index"`
    BlockHash  string `json:"block_hash"`
    Offset     int64  `json:"offset"`
    SizeBytes  int    `json:"size_bytes"`
}

type CommitRequest struct {
    SessionID  uuid.UUID    `json:"session_id"`
    NodeID     uuid.UUID    `json:"node_id"`
    TenantID   uuid.UUID    `json:"tenant_id"`
    TotalSize  int64        `json:"total_size"`
    ContentHash string       `json:"content_hash"`
    Chunks     []ChunkManifest `json:"chunks"`
}

type IngressServer struct {
    DB          *sql.DB
    SecretKey []byte
}

func (s *IngressServer) HandleInitiate(w http.ResponseWriter, r *http.Request) {
    var req struct {
        TenantID  uuid.UUID `json:"tenant_id"`
        TotalSize int64     `json:"total_size"`
        Hashes    []string  `json:"hashes"`
    }
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Malformed payload", http.StatusBadRequest)
        return
    }

    missingHashes := make([]string, 0)
    for _, h := range req.Hashes {
        hashBytes, _ := hex.DecodeString(h)
        var exists bool
        query := `SELECT EXISTS(SELECT 1 FROM cas_blocks WHERE block_hash = $1 AND tenant_id = $2)`
        _ = s.DB.QueryRowContext(r.Context(), query, hashBytes, req.TenantID).Scan(&exists)
        if !exists {
            missingHashes = append(missingHashes, h)
        }
    }

    sessionID := uuid.New()
    _, err := s.DB.ExecContext(r.Context(),
        `INSERT INTO upload_sessions (session_id, tenant_id, total_size, expected_chunks, expires_at)
        VALUES ($1, $2, $3, $4, $5)`,
        sessionID, req.TenantID, req.TotalSize, len(missingHashes), time.Now().Add(24*time.Hour))
    if err != nil {
        http.Error(w, "Session creation failure", http.StatusInternalServerError)
        return
    }
}

```

```

}

preSignedURLs := make(map[string]string)
for _, h := range missingHashes {
    preSignedURLs[h] = s.generatePreSignedURL(req.TenantID, h)
}

w.Header().Set("Content-Type", "application/json")
_ = json.NewEncoder(w).Encode(map[string]interface{}{
    "session_id":    sessionID,
    "missing_hashes": missingHashes,
    "upload_urls":   preSignedURLs,
})
}

func (s *IngressServer) HandleCommit(w http.ResponseWriter, r *http.Request) {
    var req CommitRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid Commit Manifest", http.StatusBadRequest)
        return
    }

    ctx := r.Context()
    tx, err := s.DB.BeginTx(ctx, &sql.TxOptions{Isolation: sql.LevelReadCommitted})
    if err != nil {
        http.Error(w, "Transaction initialization error", http.StatusInternalServerError)
        return
    }
    defer tx.Rollback()

    var expired bool
    err = tx.QueryRowContext(ctx,
        `SELECT (expires_at < NOW()) FROM upload_sessions WHERE session_id = $1 FOR UPDATE`, req.SessionID)
    if err != nil || expired {
        http.Error(w, "Session expired or invalid", http.StatusGone)
        return
    }

    var nextVersion int
    _ = tx.QueryRowContext(ctx,
        `SELECT COALESCE(MAX(version_number), 0) + 1 FROM file_versions WHERE node_id = $1`, req.NodeID)

    fileHashBytes, _ := hex.DecodeString(req.ContentHash)
    versionID := uuid.New()
    _, err = tx.ExecContext(ctx,
        `INSERT INTO file_versions (version_id, node_id, version_number, total_size_bytes, mime_type, content_hash)
        VALUES ($1, $2, $3, $4, 'application/octet-stream', $5, $6)`,
        versionID, req.NodeID, nextVersion, req.TotalSize, fileHashBytes, req.ContentHash)
    if err != nil {
        http.Error(w, "Version commit failure", http.StatusInternalServerError)
        return
    }

    for _, chunk := range req.Chunks {
        blockHashBytes, _ := hex.DecodeString(chunk.BlockHash)

        _, err = tx.ExecContext(ctx,
            `INSERT INTO cas_blocks (block_hash, tenant_id, size_bytes, ref_count)
            VALUES ($1, $2, $3, 1)
            ON CONFLICT (block_hash) DO UPDATE SET ref_count = cas_blocks.ref_count + 1`,
            blockHashBytes, req.TenantID, chunk.SizeBytes)
        if err != nil {
            http.Error(w, "CAS insertion failure", http.StatusInternalServerError)
            return
        }

        _, err = tx.ExecContext(ctx,
            `INSERT INTO file_manifest_blocks (version_id, chunk_index, block_hash, offset_bytes, size_bytes)
            VALUES ($1, $2, $3, $4, $5)`,
            versionID, chunk.Index, blockHashBytes, chunk.OffsetBytes, chunk.SizeBytes)
        if err != nil {
            http.Error(w, "Manifest block insertion failure", http.StatusInternalServerError)
            return
        }
    }
}

```

```

        versionID, chunk.Index, blockHashBytes, chunk.Offset, chunk.SizeBytes)
    if err != nil {
        http.Error(w, "Manifest block failure", http.StatusInternalServerError)
        return
    }
}

_, _ = tx.ExecContext(ctx, `UPDATE upload_sessions SET is_completed = TRUE WHERE session_id = $1`, req)
if err := tx.Commit(); err != nil {
    http.Error(w, "Commit confirmation failure", http.StatusInternalServerError)
    return
}

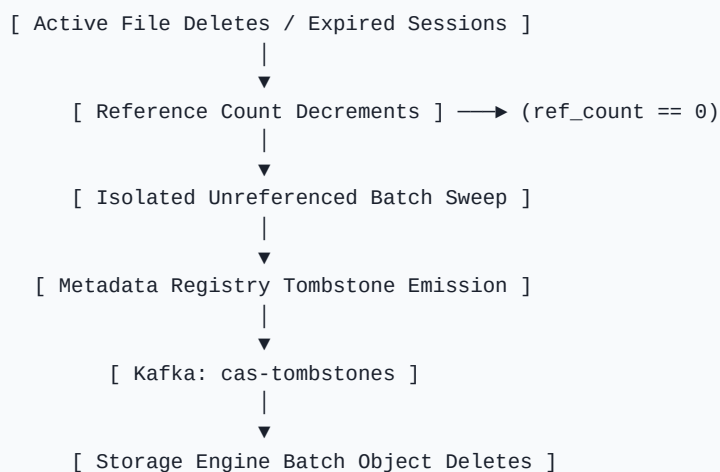
w.WriteHeader(http.StatusCreated)
_, _ = w.Write([]byte(`{"status":"COMMITTED"}`))
}

func (s *IngressServer) generatePreSignedURL(tenantID uuid.UUID, blockHash string) string {
    mac := hmac.New(sha256.New, s.SecretKey)
    mac.Write([]byte(fmt.Sprintf("%s:%s", tenantID, blockHash)))
    signature := hex.EncodeToString(mac.Sum(nil))
    return fmt.Sprintf("https://blob-storage.internal/chunks/%s?tenant=%s&sig=%s", blockHash, tenantID, signature)
}

```

6. Out-of-Band Compaction & Garbage Collection

To prevent orphaned binary data from accumulating across petabyte-scale storage tiers, the platform runs an asynchronous generational mark-and-sweep pipeline:



```

-- Step 1: Query unreferenced blocks older than the retention safety window (7 days)
CREATE TEMP TABLE unreferenced_blocks AS
SELECT cb.block_hash, cb.tenant_id
FROM cas_blocks cb
LEFT JOIN file_manifest_blocks fmb ON cb.block_hash = fmb.block_hash
WHERE fmb.block_hash IS NULL
    AND cb.created_at < NOW() - INTERVAL '7 days';

-- Step 2: Remove unreferenced entries from active registry
DELETE FROM cas_blocks
WHERE block_hash IN (SELECT block_hash FROM unreferenced_blocks);

-- Step 3: Emit deletion events to storage nodes (Kafka stream: `cas-tombstones`)

```

7. Operational Runbooks & Failure Modes

1. Incomplete Upload Leaks (Client Drop-Off)

- **Root Cause:** Clients request session allocations and upload chunks to object storage, but drop their connection prior to the manifest commit.
- **Resolution:** Object buckets enforce bucket lifecycle rules: `AbortIncompleteMultipartUpload` after 24 hours. The upload session reaper marks database sessions as `EXPIRED`, making orphan blocks eligible for the 7-day generational sweep.

2. Thundering Herd on Root Namespace Moves

- **Root Cause:** Moving a directory containing 250,000 files triggers deep recursive graph updates and invalidates cache entries across millions of connected clients.
- **Resolution:** Directory moves modify only the single targeted row and its immediate descendants via `move_directory()`. The event pipeline emits a single structural lineage update event (`NODE_MOVED`), prompting downstream clients to update the local ancestor path mapping without re-downloading child entities.

8. Quantitative Reliability & SLA Matrix

Dimension	Engineering Specification	Verification Mechanism
Durability Tier	99.999999999% (11 Nines)	Multi-region Reed-Solomon (8+4) Erasure Coding.
API Latency (Commit Manifest)	p95 < 45ms, p99 < 85ms	Connection pooling over localized distributed SQL clusters.
Token Time-To-Live (TTL)	Maximum 15 minutes	Scoped HMAC-SHA256 signatures with tenant derivation.
RPO (Recovery Point Objective)	0 seconds	Multi-region synchronous consensus logs.
RTO (Recovery Time Objective)	< 30 seconds	Automated failover across multi-availability-zone load balancers.